Digital Systems

Lecture 4

Datasheets & Programming

Mark van der Wilk

Hilary Term 2024

UNIVERSITY OF OXFORD

Department of COMPUTER SCIENCE

Following the course of past years by Mike Spivey

Datasheets

Datasheets

If we want to do assembly programming, we will need to understand the effect of each instruction.

Instruction behaviour result of hardware implementation

No way to know this, except to look it up!

- Already mentioned (lec 2) that machine code spec was in datasheet.
- Now that we're programming, let's look the ways that we can find out the effect of instructions on our computer as a whole.

All datasheets for the Microbit can be found at:

- https://spivey.oriel.ox.ac.uk/digisys/The_BBC_micro:bit

Datasheets

Designers of each part of the Microbit wrote their own datasheet.

micro:bit

Nordic nRF51822

ARM Cortex-M0

- Registers, datapath
- Thumb-based control
- Interrupt controller

- Peripherals: GPIO, UART, I2C
- 16kB RAM, 256kB Flash ROM

- LEDs, buttons via GPIO
- Accelerometer, Magnetometer via I2C
- Second processor – for USB

Contents [hide]

1 Processor core

2 Microcontroller chip

3 micro:bit board

4 Accessories

Datasheet: Pushing Buttons

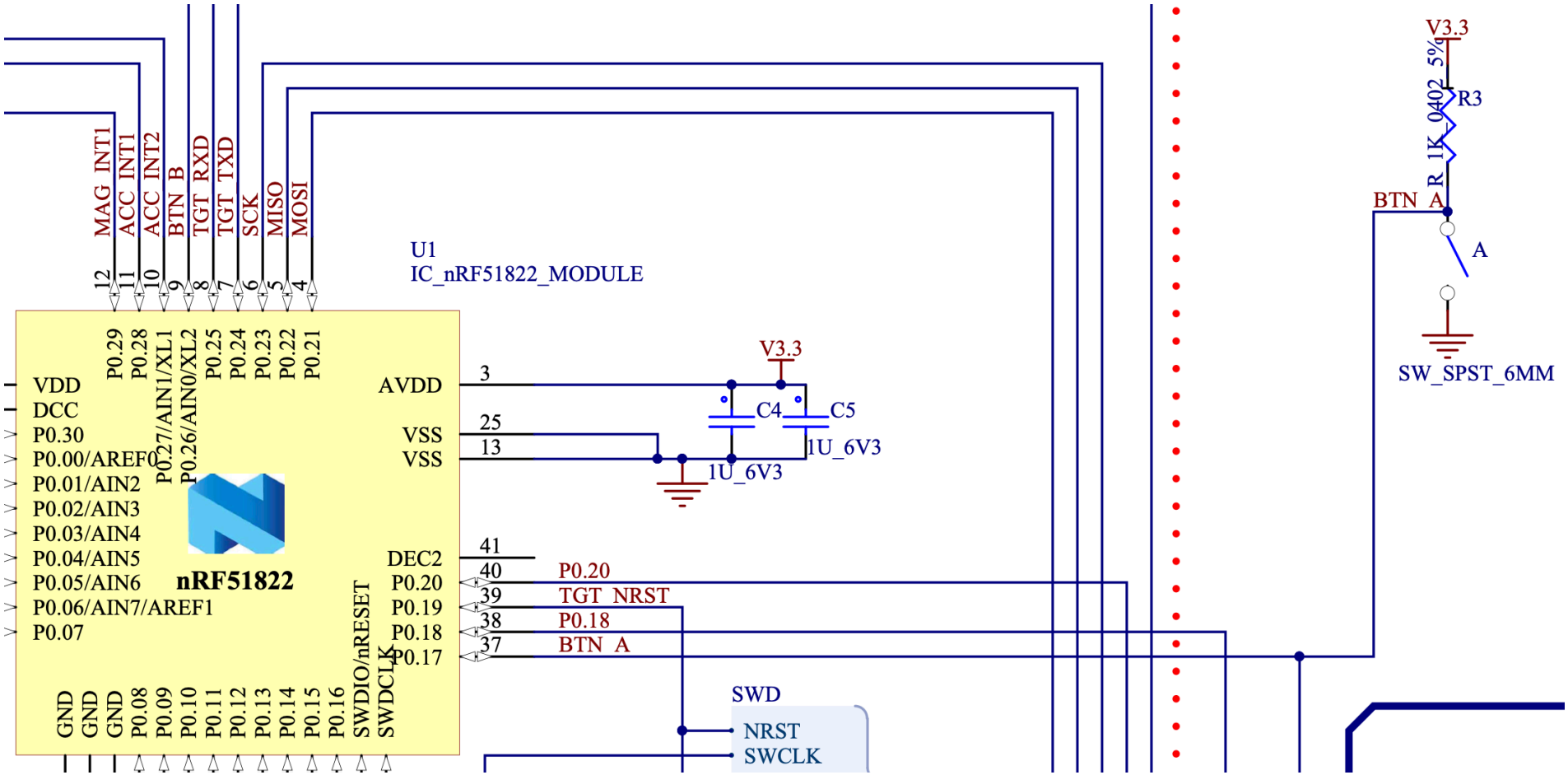
How can we determine a button is pressed?

We already said that peripherals are *memory mapped*, including GPIO.

- So we need to know which pin the button is connected to.

Microbit elecrical schematic: https://spivey.oriel.ox.ac.uk/dswiki/images-digisys/5/5b/Microbit-schematic.pdf

Datasheet: Pushing Buttons



Datasheet: GPIO

Reference manual: https://spivey.oriel.ox.ac.uk/dswiki/images-digisys/3/34/NRF51822-ref-manual.pdf

- Can find the address of the peripheral
- Can find the address of the register to **read** the state of the pin (determined by voltage put on it)

Datasheet: Sending Data through UART

Reference manual: https://spivey.oriel.ox.ac.uk/dswiki/images-digisys/3/34/NRF51822-ref-manual.pdf

- Similar process
- Can find address of TxD register
- To send data over UART

Datasheet: Different ways of Adding

How to look up allowed instructions?

We have three summaries:

- "Rainbow Chart" (included in exam)
- List of common instructions (included in exam)
- Arm Thumb Quick Reference Guide (official from ARM)

Definitive document is ARM v6-M Architecture Reference Manual.

Let's look at details of `add` instruction.

- Index: A.6 Thumb Instruction Details
- A.6.7 Alphabetical list of ARMv6-M Thumb instructions
- Various flavours of `add` instructions.

Different Flavours of Addition

Operation		§	Assembler	Updates	Action	Notes
Add	Immediate 3		`ADDS Rd, Rn, #<imm>`	N Z C V	Rd := Rn + imm	imm range 0-7.
	All registers Lo		`ADDS Rd, Rn, Rm`	N Z C V	Rd := Rn + Rm	
	Hi to Lo, Lo to Hi, Hi to Hi		`ADD Rd, Rd, Rm`		Rd := Rd + Rm	Not Lo to Lo.
	Any to Any	T2	`ADD Rd, Rd, Rm`		Rd := Rd + Rm	Any register to any register.
	Immediate 8		`ADDS Rd, Rd, #<imm>`	N Z C V	Rd := Rd + imm	imm range 0-255.

Optimise The Common Case

- Design principle: Make the common case fast, while leaving uncommon case possible.
- Amdahl's law: The maximum achievable speed-up of an optimisation is limited by how often the optimised part is used.

More Assembly Programming

Naive Multiplication in C

```
unsigned func(unsigned a, unsigned b) {
    unsigned x = a, y = b, z = 0;

    /* Invariant: a × b = x × y + z */
    while (y != 0) {
        y = y - 1;
        z = z + x;
    }

    return z;
}
```

Naive Asm Translation of Naive Multiplication in C

```
func:                       @ x in r0, y in r1
    movs r2, #0             @ z = 0
loop:
    cmp r1, #0              @ if y == 0
    beq done                @     jump to done
    subs r1, r1, #1         @ y = y - 1
    adds r2, r2, r0         @ z = z + x
    b loop                  @ jump to loop
done:
    movs r0, r2             @ return z
    bx lr
```

Which Instructions are Actually Used?

```
$ arm-none-eabi-objdump -d mul1.o
00000000 <func>:
   0:  2200      movs    r2, #0

00000002 <loop>:
   2:  2900      cmp     r1, #0
   4:  d002      beq.n   0xc <done>
   6:  3901      subs    r1, #1
   8:  1812      adds    r2, r2, r0
   a:  e7fa      b.n     0x2 <loop>

0000000c <done>:
   c:  0010      movs    r0, r2
   e:  4770      bx      lr
```

Pipelining

? Why is there an offset of 4 in relative addresses in jumps?

- `4: beq.n` would jump +2 instructions ahead.
- `a: b.n` would jump −6 instructions ahead.

Executing an instruction is more complex than previously described.

- Executing instruction is broken up into stages.
- In ARM Cortex-M0, 3 cycles needed to execute a single instruction.
- However, stages operate in *parallel*, so still 1 instruction *per* cycle.

Pipelining: Branch offset

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch executed now!

- When branch is executed, `pc` is 8.
- Want to jump two instructions ahead.
- $\Rightarrow$ Encode offset of 2 instructions!
- CPU handles multiplication by two (bit shift) before adding to `pc`.

How Good is Our Code?

Good assembly programmers (compiler, mostly) should produce *fast* code.

Instruction timings completely predictable (unlike modern archs):

- Usually: One cycle per instruction.
- Branch: Taken branch takes 2 cycles *extra* (3 total).
- Load / store: Plus one cycle.

Find number of cycles:

- Can measure (oscilloscope, see labs)
- Count instructions (7 cycles)

? Why do branches take 3 cycles?

Pipelining: Normal Operation

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch not taken
6	10	`b`	`adds`	`subs`	next instruction as normal
7	12	`movs`	`b`	`adds`	

Pipelining: Branch Taken

cycle	pc	Fetch	Decode	Execute	
1	0	`movs`			
2	2	`cmp`	`movs`		
3	4	`beq`	`cmp`	`movs`	
4	6	`subs`	`beq`	`cmp`	
5	8	`adds`	`subs`	`beq`	branch taken!
6	12	`movs`	`adds X`	`subs X`	do not execute!
7	14	`bx`	`movs`	`adds X`	do not execute!
8	16	`...`	`bx`	`movs`	as normal

Can We Do Better?

Solutions:

- Better algorithm (e.g. $O(\log n)$).
- "Squeeze" loop.

Optimisation

```
loop:
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
loop1:
    subs r1, r1, #1     @ y = y - 1
    adds r2, r2, r0     @ z = z + x
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
    b loop1             @ jump to loop1
done:
```

Optimisation

Swap and simplify.

```
loop:
    cmp r1, #0          @ if y == 0
    beq done            @     jump to done
loop1:
    adds r2, r2, r0     @ z = z + x
    subs r1, r1, #1     @ y = y - 1
    cmp r1, #0          @ if y != 0
    bne loop1           @     jump to loop1
done:
```

Optimisation

Let subs set the condition codes.

```
loop:
    cmp r1, #0              @ if y == 0
    beq done                @     jump to done
loop1:
    adds r2, r2, r0         @ z = z + x
    subs r1, r1, #1         @ y = y - 1
    bne loop1               @ if y != 0, loop
done:
```

- 5 cycles per iteration!
- Would be `do..while` loop in C.

Loop Unrolling

Duplicate the loop body n times. m

```
loop1:
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    ...
```

```
    adds r2, r2, r0
    subs r1, r1, #1
    beq done
    adds r2, r2, r0
    subs r1, r1, #1
    bne loop1
done:
```

- 3.5 cycles per iteration!

Observations on Assembly Programming

- Writing optimised asm code is labourious! Small changes in details of program impact many other lines (e.g. change in register!)
- Good compilers can generate very efficient assembly code from C code.
  - E.g.: It's hard for us to use higher registers. Requires knowing a lot of these quirks of assembly, which instructions can use high registers, and how. Compilers can do this well.
- Remember Amdahl's law!
  - It's good to know *how* to write a very fast, tight loop, but no need to spend all your time doing this for *every* bit of code.
  - Focus on where it matters.

Recap

Looking up things in datasheets

Instruction set restrictions

... exist for a reason and are specified in the datasheet.

Pipelining

- Instructions are executed over multiple clock cycles in parallel.
- Complicates branch instructions, and makes them slower.

Determining execution time of assembly program

More complex assembly programming, and optimisation